Interview Notes on Elastic Block Store (EBS) for EC2 Instances 🚀

---

1. What is an EBS Volume? 📦

- **Elastic Block Store (EBS)** is a **network drive** that can be **attached to EC2 instances** for persistent data storage. 💻
- Unlike **instance store volumes**, EBS allows data to persist even after the EC2 instance is terminated. 📁

2. Key Features of EBS 💡

- **Persistence**: Data remains intact after an EC2 instance is terminated, making it highly useful for storing critical data. 🔄
- **Network Drive**: EBS volumes work over the **network** (not physically attached) to communicate with EC2 instances, meaning there can be **slight latency** in data transfer. ⚡
- **Single Instance Attachment**: At a time, an EBS volume can only be attached to **one EC2 instance** within a specific **Availability Zone (AZ)**. ❌🔗

---

3. EBS Volume & Availability Zones 🌍

- When creating an EBS volume, it's **bound to a specific AZ** and cannot be attached across AZs (e.g., an EBS volume in `us-east-1a` can't be attached to an EC2 instance in `us-east-1b`). 🌐
- If needed, **snapshots** of the EBS volume can be taken to **move volumes** across different AZs.

---

4. Provisioning & Billing 💰

- **Provisioning**: You need to decide the **capacity** (in GBs) and the number of **IOPS** (Input/Output operations per second) when creating an EBS volume. 📊
- **Billing**: You are billed for the **provisioned capacity** (i.e., the amount of storage you've allocated), not the actual usage. 📈
    - You can **increase** the size or performance (IOPS) as needed over time.

---

5. Diagram of EBS Usage with EC2 💻

```
   US-East-1a
      |
+----------------+          +----------------+
| EC2 Instance 1 |<-------->|   EBS Volume 1 |
+----------------+          +----------------+
```

```
        |
    +----------------+          +----------------+
    | EC2 Instance 2 |<-------->|   EBS Volume 2 |
    +----------------+          +----------------+

(Each EC2 instance can have its own EBS volume attached)
```

➡️ EBS volumes can be detached and attached to different EC2 instances across different times!

- **Multiple Volumes on One EC2**: You can attach **multiple EBS volumes** to a single EC2 instance, similar to plugging multiple USB sticks into a computer. 💾

---

6. Detaching and Attaching Volumes 🔄

- EBS volumes can be **detached** from one EC2 instance and **attached** to another **quickly**—helpful in **failover** scenarios. ⚠️

---

7. Delete on Termination Flag ❌🛑

- **Delete on Termination** (DoT) is a **flag** that determines whether the EBS volume should be **deleted** when the EC2 instance is terminated.
  - By default, **root volumes** (the volume containing the EC2 instance's OS) have **DoT enabled**.
  - **Additional EBS volumes** (non-root) are **not deleted** by default when the EC2 instance is terminated. 🧰

---

8. Important Use Cases 🛠️

- **Preserving Data**: If you want to ensure that your data **is not lost** after the termination of an EC2 instance, you can disable the **DoT** flag on EBS volumes. For example, if you want to keep logs or other essential data after shutting down an EC2 instance, disabling DoT ensures you can later access those volumes. 📄

---

9. Summary 📝

- **EBS Volume = Network Drive** (Think of it as a **cloud USB stick**).
- **Provisioned Capacity**: You set the size and IOPS.
- **Single Instance Attachment**: One volume per EC2 instance, but multiple volumes per instance are possible.
- **Snapshotted Volumes**: Can be moved across AZs using snapshots.

- **Delete on Termination**: Default behavior for root volumes, can be modified for non-root volumes.

Final Note for Interview Preparation 🎯

- Be familiar with the concept of **EBS volumes** and their **connection** to **EC2 instances**.
- Know the implications of **EBS volume detachment/attachment**, **provisioning**, and **billing**.
- Understand the **Delete on Termination** flag and its **real-world use cases** (e.g., preserving important data).

Interview Notes on EBS Volumes: Management and Termination 🚀

---

1. Navigating EBS Volumes in AWS Console 🖥️

- **Step 1**: Navigate to the EC2 instance and go to the **Storage tab** to see attached volumes.
  - You'll see a **root device** and **block devices** associated with the instance. 🏷️
- **Step 2**: Click on the volume to be taken to the **Volumes interface** in AWS.
  - This shows details like the volume's **status** (e.g., "in use") and the **attached instance**.

---

2. Creating and Attaching Additional EBS Volumes 🛠️

- **Step 1**: Click on **Create Volume** in the AWS EC2 Console.
  - Choose the **Volume Type** (e.g., **GP2** or **GP3**).
  - Specify the **Size** (e.g., 2 GB) and the **Availability Zone (AZ)** (e.g., `eu-west-1b` for consistency).

  Example:
  If your instance is in `eu-west-1b`, the new volume must be created in the same AZ due to EBS volume-AZ binding. 🌍
- **Step 2**: Once the volume is created, it will be in an **available** state.
  - **Attach Volume**: Choose the EC2 instance to attach it to.
  - The volume is now **attached** to your EC2 instance, as seen in the EC2 console under **Block Devices**. 🖧

---

3. Verifying Multiple EBS Volumes on EC2 Instance 🔄

- **Step 1**: After attachment, navigate to **Storage** → **Block Devices** in the EC2 console.

- You will see the list of all attached volumes.
  Example:
- **Volume 1**: 8 GB
- **Volume 2**: 2 GB
  These volumes are now visible for usage. 💾

---

4. Understanding EBS Volumes Across Different AZs 🌐

- EBS volumes are **bound to specific Availability Zones (AZs)**.
  - **Example**: If an EC2 instance is in `eu-west-1b`, the volume must be in the same AZ (`eu-west-1b`).
- **Scenario**: If you create a volume in a different AZ (`eu-west-1a`), it **cannot be attached** to the EC2 instance in `eu-west-1b`. ⚠️
  - This demonstrates the **AZ restriction** for EBS volumes. 📍

---

5. Deleting EBS Volumes 🗑️

- **Deleting Volumes**: You can easily **delete volumes** with a few clicks:
  - Select the volume, click on **Actions**, and choose **Delete Volume** to remove it. 👈
  - This is part of the flexibility and speed of cloud resources. ⚡

---

6. Understanding the "Delete on Termination" Attribute 🛑

- Each **EBS volume** has a **Delete on Termination** flag that controls whether the volume is deleted when the EC2 instance is terminated.
  - **Root Volumes** (the main boot volume) **default to "Delete on Termination" = Yes**. 🛑
  - **Non-root Volumes** (additional volumes) **default to "Delete on Termination" = No**. ❌

---

7. Real-World Scenario: EC2 Instance Termination ⏳

- **Scenario**: If you **terminate the EC2 instance**, the following happens:
  - **Root Volume (8 GB)**: Since it has **Delete on Termination = Yes**, it will be deleted when the instance is terminated. ❌💾
  - **Attached Volume (2 GB)**: Since it has **Delete on Termination = No**, it will not be deleted and remains in the **available** state. ✅
  - Once the instance is terminated:
    - The root volume (8 GB) disappears. 🚩

- The non-root volume (2 GB) remains. 🏆

---

8. Deleting EC2 Instance and Its Effects on Volumes ⚠️

- After terminating the EC2 instance:
  - You can see in the **EBS Volumes** section that the **terminated instance's root volume** is no longer listed.
  - The **non-root volumes** remain in the available state, ready to be detached or attached to another EC2 instance. 🔄

---

9. Key Takeaways for Interview 📚

- **Volume-AZ Binding**: EBS volumes are always **bound** to a specific **Availability Zone**.
- **Volume Attachment**: An EBS volume can only be attached to one EC2 instance at a time.
- **Delete on Termination**: The **root volume** of an EC2 instance has **Delete on Termination = Yes** by default, while **non-root volumes** have **Delete on Termination = No**.
- **Flexibility**: You can easily **create**, **attach**, and **delete** EBS volumes within seconds, showcasing the flexibility of cloud computing. ⏱️

Final Note for Interview Preparation 🎯

- Be ready to explain **EBS volume attachment/detachment**, **AZ binding**, and **termination behavior** with real-world examples.
- Understand the **Delete on Termination flag** and its implications for EC2 instance lifecycle and data persistence. 🔑

Interview Notes on EBS Snapshots 🔄

---

1. What is an EBS Snapshot? 📸

- An **EBS Snapshot** is essentially a **backup** of your EBS volume taken at any given point in time. ⏳
- It is a **point-in-time** copy that can be used for:
  - **Backup and Restore**: Recover lost or corrupted data.
  - **Migration**: Transfer data across different Availability Zones (AZs) or even AWS Regions.

---

2. Snapshot and Volume Detachment 🔗

- **Detachment**: While it's **not mandatory** to **detach** the EBS volume from the EC2 instance to take a snapshot, it’s **recommended** for consistent backups. 🛠️
  - Detaching ensures that the snapshot captures **data consistency** without any data being written to the volume during the snapshot.

---

3. Cross-AZ and Cross-Region Snapshots 🌍

- **Cross-AZ Transfer**: You can **copy EBS snapshots across Availability Zones (AZs)** within the same region. This is helpful for **disaster recovery** or migrating workloads. ⚙️
  - **Example**: If an EC2 instance has an EBS volume in `US-EAST-1A`, you can take a snapshot and restore it in `US-EAST-1B`. 🔄
- **Cross-Region Transfer**: EBS snapshots can also be copied across **different AWS regions**. This is useful for **region-based redundancy** or backup purposes. 🌐

---

4. EBS Snapshot Archive Tier 📦

- **Snapshot Archive**: You can move an EBS snapshot to an **archive tier**, making it **up to 75% cheaper**.
  - This is ideal for **long-term storage** where you don’t need immediate access to the snapshot. 📉
  - **Recovery Time**: Restoring an archived snapshot will take **24 to 72 hours**. So, this is **not an immediate recovery** solution. ⏳

---

5. EBS Snapshot Recycle Bin 🗑️

- **Recycle Bin for Snapshots**: If you accidentally **delete an EBS snapshot**, it doesn’t get permanently removed immediately. Instead, it goes into a **Recycle Bin**. 🔄
  - You can restore the snapshot from the Recycle Bin, allowing recovery from **accidental deletions**.
- **Retention Settings**: You can configure the **retention period** for snapshots in the Recycle Bin, from **1 day to 1 year**. ⏳

---

6. Fast Snapshot Restore (FSR) ⚡

- **Fast Snapshot Restore** (FSR): This feature allows you to **pre-warm** a snapshot, so that there’s **no initialization latency** when the snapshot is first used. 🚚
  - Especially useful for **large snapshots** that need to be used immediately after restore.

- **Cost Consideration**: This feature is **expensive**, so use it judiciously based on your performance requirements. 💸

---

7. EBS Snapshot Use Cases 🌟

- **Backup and Disaster Recovery**: EBS snapshots are crucial for ensuring that you can **recover** data after an incident.
- **Data Migration**: EBS snapshots allow for **easy migration** of data across AZs and regions.
- **Long-Term Storage**: Using the **Archive Tier**, you can store snapshots at a low cost for long-term backup.
- **Quick Restores**: The **Fast Snapshot Restore** feature allows rapid recovery when you need it the most, especially for large datasets.

---

8. Summary for Interview Preparation 📚

- **Snapshot Basics**: EBS snapshots are backups of EBS volumes, taken at specific points in time.
- **Copying Snapshots**: You can copy snapshots across AZs or AWS regions for migration or redundancy purposes.
- **Snapshot Archive**: Moving snapshots to the archive tier saves money but comes with delayed restore times.
- **Recycle Bin**: Deleted snapshots are not permanently gone; they go into a Recycle Bin for recovery within the retention period.
- **FSR**: Fast Snapshot Restore allows for faster initialization but comes with a higher cost.

Final Note for Interview 🎯

- Be prepared to discuss **EBS snapshot features** such as **Archive Tier**, **Recycle Bin**, and **Fast Snapshot Restore** in detail.
- Highlight **cost** considerations and **use cases** for snapshots in backup, migration, and performance-sensitive scenarios. 🌟

Interview Notes on EBS Snapshots: Management and Recovery 🔄

---

1. Creating a Snapshot 📸

- To create a snapshot of an EBS volume:
  1. Go to **Volumes** → Select the EBS volume (e.g., 2 GB GP2 volume).
  2. Click **Actions** → **Create Snapshot**.

3. Add a **description** (e.g., `DemoSnapshot`) and click **Create Snapshot**.

This will create a backup of your EBS volume at a **point-in-time**.

---

2. Managing Snapshots 🛠️

- After creating a snapshot, you can manage it by going to the **Snapshots** section in the AWS Console. 📂
  - You can see details like:
    - **Snapshot status** (Completed, Available).
    - **Snapshot ID**.
    - **Size** and **description**.

---

3. Copying Snapshots Across Regions 🌍

- **Copy Snapshot**: Snapshots can be copied to different AWS regions. This is useful for **disaster recovery** strategies or maintaining cross-region backups. 🔄
  - Right-click on the snapshot → **Copy Snapshots** → Choose the destination region.
  - This ensures that your data is backed up in another **region** for added redundancy. ⚡

---

4. Creating a New Volume from a Snapshot 📦

- **Restore from Snapshot**: You can create a new EBS volume from a snapshot.
  Steps:
  1. Go to **Snapshots** → Select the snapshot.
  2. Click **Actions** → **Create Volume from Snapshot**.
  3. Choose the **Size** (e.g., 2 GB GP2), the **Availability Zone (AZ)** (e.g., `eu-west-1b`).
  4. Optionally, you can **encrypt** the volume or add **tags** for identification.
- **Result**: A new volume will be created from the snapshot in the selected AZ.
  This allows you to **copy** EBS volumes across different AZs for **load balancing** or **failover**. ⚖️

---

5. Working with the Recycle Bin 🗑️

- **Recycle Bin for Snapshots**: The **Recycle Bin** protects EBS snapshots (and Amazon Machine Images, AMIs) from **accidental deletion**.

  Steps to Use Recycle Bin:

  1. Create a **Retention Rule** to define how long snapshots should remain protected after deletion (e.g., 1 day).

2. You can choose whether the rule is **locked** or **unlocked**. An **unlocked rule** allows deletion of the rule at any time.

Example: Create a rule called `DemoRetentionRule`, apply it to all snapshots, and retain them for **1 day**. 📅

- When you delete a snapshot, it **doesn't get permanently deleted** right away. It goes into the **Recycle Bin**, where it can be **recovered**. 🔄

  1. If you delete a snapshot, you can go to **Recycle Bin** → **Restore** to bring it back into the **Snapshots** section.

---

6. Storage Tiers for Snapshots 💲

- **Standard Storage Tier**: When you first create a snapshot, it is stored in the **Standard Storage Tier**, which is designed for frequent access.
- **Archive Tier**: If you want to **reduce costs**, you can move the snapshot to an **Archive Storage Tier**, which is **75% cheaper**. 📉
  - **Restore Time**: If you need to restore an archived snapshot, expect a wait of **24 to 72 hours**. This is useful for long-term storage where you don't need immediate access.

---

7. Deleting and Recovering Snapshots 🗑️ ➡️ 🔄

- **Deleting a Snapshot**: You can **delete** snapshots directly from the console, but before **March 2021**, once deleted, the snapshot was permanently gone. ⚠️
  - Now, thanks to the **Recycle Bin**, deleted snapshots can be recovered. 🛠️
- **Restoring from Recycle Bin**: If you accidentally delete a snapshot:
  - Go to the **Recycle Bin**.
  - Select the deleted snapshot and click **Recover Resources** to restore it back into your Snapshots list. ✅

---

8. Snapshot Use Cases 🏗️

- **Backup**: Use snapshots for **regular backups** of your EBS volumes. They provide a point-in-time recovery solution. 🛡️
- **Disaster Recovery**: Copy snapshots across regions to **protect** against regional outages or failures. 🌍
- **Cross-AZ Data Distribution**: Restore snapshots in different AZs to enhance availability and fault tolerance. ⚖️
- **Cost Optimization**: Use the **Archive Tier** to save costs on snapshots that are not frequently accessed but need to be retained for compliance or backup purposes. 💰

- **Accidental Deletion Recovery**: With the **Recycle Bin**, safeguard against human errors and accidental deletions. 🛑

---

9. Summary for Interview Preparation 📚

- **Snapshots** are **backups** of EBS volumes and can be used to **restore** data or create new volumes.
- You can **copy snapshots** to **different regions** for **disaster recovery** or **cross-region backups**.
- The **Recycle Bin** ensures protection from accidental snapshot deletion, allowing easy recovery.
- **Storage Tiers** (Standard vs Archive) help manage cost, with archived snapshots being cheaper but slower to restore.
- Be aware of **retention rules** and their role in keeping snapshots protected for specified periods.

Final Note for Interview 🎯

- Make sure to explain **how snapshots work** (creation, restoration, and cross-region use).
- Emphasize the **cost savings** with Archive Storage and the **importance** of using **Recycle Bin** for data protection.
- Be ready to discuss **real-world scenarios** where you would use EBS snapshots for **disaster recovery**, **migration**, or **backup strategies**. 🌟

Interview Notes on Amazon Machine Images (AMIs) 🖼️

---

1. What is an AMI? 💡

- **AMI (Amazon Machine Image)** is a **pre-configured template** that contains the information required to launch an **EC2 instance**.
- It includes:
  - **Operating System** (e.g., Linux, Windows).
  - **Software configurations** (e.g., web server, database).
  - **Application settings** (e.g., monitoring tools, custom scripts).

  In simple terms, an AMI is a **snapshot** of your instance that includes both the OS and any additional software installed on it. 💻

---

2. Types of AMIs 🌍

- **Public AMIs**: These are **pre-built** by AWS and available for everyone to use.
  - **Example**: Amazon Linux 2 AMI is popular and widely used. 🐧

- **Custom AMIs**: You can create your own AMI based on a configured EC2 instance. This allows you to package your customized environment and use it to launch multiple EC2 instances in the future. 📦
    - **Faster Boot Times**: Since the software is pre-packaged, your EC2 instances will boot and configure faster. ⚡
- **Marketplace AMIs**: These are AMIs created by **third-party vendors** and offered on the **AWS Marketplace**. These might include specialized configurations or software, often sold as a product. 💼
    - You can **buy** or **license** these AMIs for specific applications, saving you time on setup. 💰

---

3. Creating a Custom AMI 🛠️

- **Steps to Create a Custom AMI**:
    1. **Launch an EC2 Instance**: Start with a base EC2 instance (e.g., a public AMI or your own custom EC2 setup).
    2. **Customize the Instance**: Install required software, configurations, and settings.
    3. **Stop the Instance**: Ensure data integrity by stopping the instance before creating the AMI.
    4. **Create an AMI**: This will create a snapshot of the instance's **EBS volumes** and capture all the changes made to the EC2 instance. 📸
- **Why Create Your Own AMI?**:
    1. **Consistency**: Ensure all EC2 instances launched from the AMI have the same configuration and setup.
    2. **Faster Deployments**: Once the AMI is created, you can quickly launch instances with your pre-configured settings. 🚀

---

4. Copying AMIs Across Regions 🌍

- **Cross-Region Use**: AMIs are tied to specific regions, but you can **copy** an AMI from one region to another. This allows you to use the same configuration in different AWS regions. 🔄
    - **Example**: You launch an EC2 instance in `US-EAST-1A`, create an AMI, and then copy the AMI to `US-EAST-1B` to launch the same instance in a different region. 🌐

---

5. AMI Lifecycle 🔄

- **From EC2 to AMI**:

    - **Launch EC2 instance** → **Customize** it → **Stop the instance** → **Create AMI** → Launch more instances using that AMI. 🛠️

- **Behind the Scenes**:
    - When you create an AMI, **EBS snapshots** are created in the background to capture the current state of your volumes. 📦

---

6. AWS Marketplace AMIs 🛒

- **Third-Party AMIs**: Vendors can create **pre-configured AMIs** with their software or applications and sell them via the **AWS Marketplace**.
    - **Example**: A vendor could create an AMI with **pre-installed databases** or **web server configurations**, and sell it to save time for users.
    - You can also **create your own AMI** and sell it in the AWS Marketplace, offering other businesses the chance to purchase your custom AMI. 💼

---

7. Example Workflow 💻

- **Region-Specific Custom AMI Workflow**:
    1. **Launch EC2 instance in `US-EAST-1A`**.
    2. **Customize** the instance (install software, configure settings).
    3. **Stop** the instance to ensure consistency.
    4. **Create AMI** from this EC2 instance.
    5. **Copy AMI to `US-EAST-1B`** (if needed).
    6. **Launch instances** from the AMI in any region (e.g., `US-EAST-1B`).

---

8. Key Points for Interview 📝

- **Definition**: An **AMI** is a pre-configured template used to launch EC2 instances with specific OS, applications, and configurations.
- **Types**:
    - **Public AMIs** (AWS provided).
    - **Custom AMIs** (user-defined).
    - **Marketplace AMIs** (third-party software, for purchase).
- **Creating a Custom AMI** involves stopping an EC2 instance, creating an image (AMI), and using EBS snapshots for storage.

- **Cross-region Copying** of AMIs allows you to launch instances in multiple regions based on the same template.

Final Note for Interview 🎯

- Make sure you understand the **process** of creating and using **custom AMIs** for scaling EC2 deployments efficiently.
- Be able to explain **how AMIs help in reducing setup time** and provide **consistency** across multiple EC2 instances.
- Highlight the importance of **copying AMIs across regions** for **global deployment** and redundancy.

Here's a detailed and simplified breakdown of the process of creating an **Amazon Machine Image (AMI)** from an EC2 instance, along with all key steps, using emojis and ASCII diagrams where necessary.

💻 Step-by-Step Process for Creating an AMI:

---

1. Launch the EC2 Instance

- **Choose Amazon Linux 2** for the instance type.
- **Select the Instance Type:**
  - Choose `t2.micro` (or any size based on your requirement).
- **Choose a Key Pair:**
  - Select an existing key pair (like `easy-to-draw`).
- **Network Settings:**
  - Select an **existing security group** from the previous launch wizard.
- **Storage:**
  - Keep the default storage settings.
- **Advanced Details:**
  - Go to **User Data** and paste the script below (but skip the last line):

```
#!/bin/bash
yum update -y
yum install httpd -y  # Install Apache HTTPD (Web Server)
systemctl start httpd  # Start the service
```

- The **last line** (systemctl command) is **not copied** to avoid creating an index file, just the installation and setup of Apache.

Why? Because you're preparing the base AMI without creating a sample file at this point.

---

2. Wait for Instance Initialization (Important!)

- Once the instance is running, it may take **1-2 minutes** for the **EC2 user data script** to execute completely.
- **Don't rush!** The instance will **not** respond to HTTP requests immediately.
- If you try to access it before the script runs, you'll get a **connection error**.
- After a couple of minutes, refreshing will show you the Apache **test page**. ✅

---

3. Create the AMI (Amazon Machine Image)

- After your instance is ready and running:
  - **Right-click** on the instance.
  - Choose **"Image and Templates" → "Create Image"**.
- Name it, e.g., **"demo-image"**.
- Keep default settings and click **Create Image**.
- **Patience is key**: the AMI status will be **Pending** until it's created.

---

4. Launch New Instance from the AMI

- Once your AMI is created (status: **"available"**):
  - Go to **AMIs** (in the left sidebar).
  - Select **My AMIs** and pick your **"demo-image"**.
  - Choose the instance type (e.g., `t2.micro`) and configure the settings:
    - Select the **same key pair** and **security group** from before.
- **User Data** (for new instance):
  - **Copy only the first 3 lines** of the script.
  - Skip the **last line** (as HTTPD is already installed in the AMI, no need to reinstall).

```
#!/bin/bash
echo "Starting Instance from AMI"  # Optional: Log output
echo "Hello World from the new instance!"
```

- **Launch the Instance**!

---

5. Access the Instance Quickly

- Once your new instance is running:
  - Use its **public IP address** to access it via HTTP.
- **Result**:
  - You'll see a **"Hello World"** page from Apache.
  - **Faster startup** compared to the initial setup since HTTPD is already pre-installed in the AMI.

Benefits of Using AMIs 🚀:

- **Speed**: Reduces setup time (Apache is already installed).
- **Consistency**: Allows replication of environments.
- **Customization**: AMIs can include essential software, security settings, or configurations.
- **Use Case**: Create an AMI after setting up a base environment (e.g., LAMP stack, security patches), and then launch future instances from that AMI, saving time.

🛑 Clean-Up: Terminate the Instances

- Once done testing, you can **terminate** both instances:
  - The **original EC2 instance** created from scratch.
  - The **new instance** created from your AMI.

Visual Representation:

```
   EC2 Instance ➡️ User Data Script ➡️ Install Apache (HTTPD) ➡️ Test Page (Hello
World)


   ┌──────────────────────────────────────┐
   |              EC2 Instance            |
   |  Launch EC2, install HTTPD via       |
   |  user data, wait for the instance    |
   |  to be ready.                        |
   └──────────────────────────────────────┘

            ↓

  ┌──────────────────────────────────────┐
```

```
|           Create an AMI (Image)      |
|  Package EC2 instance as a reusable  |
|  Amazon Machine Image (AMI).         |
└──────────────────────────────────────┘

          ↓

┌──────────────────────────────────────┐
|    Launch New Instance from AMI      |
|  Faster startup, HTTPD already set!  |
└──────────────────────────────────────┘
```

Quick Recap:

1. **Launch EC2** and configure settings.
2. **Run User Data Script** to install Apache.
3. **Create an AMI** from the running instance.
4. **Launch New EC2 Instance** from the AMI (with faster setup).
5. **Terminate Instances** after testing.

Here's a breakdown of the important points from this lecture about **EC2 Instance Store** and its use cases, formatted into notes for your interview prep:

⚡ EC2 Instance Store: High-Performance Storage

---

1. What is EC2 Instance Store?

- **Hardware Disk Attached to EC2 Instance:**
  - EC2 Instances are **virtual machines** running on physical servers in the cloud.
  - Some EC2 instances have direct access to **hardware disks** physically connected to the underlying server.
  - This disk storage is called **EC2 Instance Store**.

2. Why Use EC2 Instance Store?

- **Super High I/O Performance:**
  - EC2 Instance Store is great for **extremely high disk performance**.
  - Ideal for workloads requiring fast **read/write throughput**.

3. Performance of EC2 Instance Store vs. EBS

- **Instance Store Performance:**
  - It supports **high IOPS (Input/Output Operations Per Second)**.
  - Some **I3 instance types** can handle up to **3.3 million read IOPS** and **1.4 million write IOPS**.
- **Comparing with EBS (Elastic Block Store):**
  - For example, an **EBS volume of type GP2** can handle around **32,000 IOPS**.
  - So, **Instance Store** provides much **higher I/O performance** than EBS volumes.

⚙️ **Key Point**: **If you need ultra-fast I/O performance**, think about using EC2 Instance Store.

4. Use Cases for EC2 Instance Store:

- **Temporary, Scratch, or Cache Data:**
  - **Great for buffer**, **temporary files**, **caching**, or **scratch data**.
  - Best used when the data **doesn't need to persist** long-term.
- **NOT for Long-Term Storage**:
  - **Ephemeral storage** – Data is lost if the instance is stopped or terminated.
  - Not suitable for **long-term** or **durable** data storage.

⚠️ **Warning**: If the EC2 instance stops or fails, **data will be lost**.

---

5. Responsibilities with EC2 Instance Store:

- **Backups & Replication**:
  - If you use EC2 Instance Store, **it is your responsibility** to ensure data is backed up and replicated as needed.

🛑 **Important**: You cannot rely on EC2 Instance Store as the **primary storage** for critical data.

---

6. What Happens if the Instance Fails?

- **Risk of Data Loss**:
  - If the underlying **hardware server fails** (where the EC2 instance is running), the attached EC2 Instance Store will fail as well.
  - **Data loss risk** is high, so ensure that any important data is **backed up** elsewhere (e.g., **EBS** or **S3**).

📝 Recap:

- **Use EC2 Instance Store for**:
  - **High-performance disk operations** (e.g., caching, scratch data).
  - Temporary **non-durable storage**.
- **Avoid EC2 Instance Store for**:
  - **Long-term storage** of important data.

Visual Diagram:

```
EC2 Instance Store

┌──────────────────────────────────┐
|      High-Performance Storage     |  ➡️ Fast IOPS
|   (ephemeral, local storage)      |
└──────────────────────────────────┘

              ↑
 Great for cache, buffers, or temporary data!
              ↓
**Not for long-term, durable storage!**
```

Key Exam Tip 📚:

- **In the exam**, when you see **very high-performance IOPS** or **local storage** associated with an EC2 instance, think **EC2 Instance Store**. It provides fast read/write operations but doesn't persist if the instance is stopped or terminated.

Here's a detailed breakdown of **EBS Volumes** and their different types, along with key takeaways for your interview preparation. I've added emojis and ASCII diagrams for clarity.

💾 EBS Volumes: Types and Use Cases

---

1. Different EBS Volume Types:

EBS (Elastic Block Store) volumes come in **6 types** that can be grouped into **3 categories**:

🔋 SSD-Based Volumes (Solid-State Drives):

1. **gp2 (General Purpose SSD)**:

    - **Balanced** price/performance.
    - Ideal for most workloads (e.g., boot volumes, dev/test environments).
    - **Size**: 1 GB to 16 TB.
2. **gp3 (Next-Gen General Purpose SSD)**:
    - Newer, **faster**, and more **cost-efficient** than gp2.
    - Independent scaling of **IOPS** and **throughput**.
        - **IOPS**: Baseline 3,000, up to 16,000.
        - **Throughput**: Baseline 125 MB/s, up to 1,000 MB/s.
3. **io1 & io2 (Provisioned IOPS SSD)**:
    - **Highest performance** for **mission-critical** workloads (e.g., databases).
    - **Independently provisioned IOPS**.
    - io1: Max **64,000 IOPS**, size 4-16 TB.
    - io2: Max **256,000 IOPS**, size up to **64 TB**.

💡 HDD-Based Volumes (Hard Disk Drives):

1. **st1 (Throughput Optimized HDD)**:
    - **Low-cost**, high **throughput** for workloads like **big data**, **data warehousing**, and **log processing**.
    - **Max throughput**: 500 MB/s
    - **Max IOPS**: 500
2. **sc1 (Cold HDD)**:
    - **Lowest cost**, **infrequently accessed** storage (e.g., archival data).
    - **Max throughput**: 250 MB/s
    - **Max IOPS**: 250

---

2. How to Define an EBS Volume?

Key factors that define an EBS volume:

- **Size**: From 1 GB to 64 TB.
- **Throughput**: Speed of data transfer.
- **IOPS** (Input/Output Operations Per Second): Speed of read/write operations.

---

3. Boot Volumes: Which EBS Types Can Be Used?

- Only **gp2**, **gp3**, **io1**, and **io2** can be used as **boot volumes** (for the OS).
- **st1** and **sc1 cannot** be used as boot volumes.

📊 gp2 vs gp3 (General Purpose SSD)

Feature	gp2 (Older)	gp3 (Newer)
IOPS	**Linked to volume size** (max 16,000)	**Independent scaling** (up to 16,000)
Throughput	Linked to volume size (up to 250 MB/s)	Up to **1,000 MB/s** independently from IOPS
Cost Efficiency	Less cost-effective than gp3	**More cost-effective** with better performance

Key takeaway: **gp3** is **more cost-efficient** and provides independent scaling of IOPS and throughput, making it better for modern workloads compared to **gp2**.

🚀 Provisioned IOPS Volumes (io1 & io2)

- **io1**:
  - For **critical business applications** needing sustained performance (e.g., database workloads).
  - **Max IOPS**: **64,000** for standard EC2 instances, **32,000** for others.
  - **Size**: Between 4 GB to 16 TB.
- **io2**:
  - **Sub-millisecond latency** for ultra-high performance workloads.
  - **Max IOPS**: **256,000**.
  - **Size**: Up to **64 TB**.
  - **1,000 IOPS per GB**.

Key takeaway: **io1 & io2** are **best for database workloads** or anything that needs **consistent, high performance**.

🔎 st1 & sc1 for Cost-Effective Storage

- **st1**:
  - **Low-cost HDD** volume for **throughput-intensive** workloads.
  - **Great for**: Big data, data warehousing, and log processing.
  - **Max throughput**: 500 MB/s, **Max IOPS**: 500.
- **sc1**:

- **Lowest-cost HDD** for **cold storage** (infrequently accessed).
- **Best for**: Archival data.
- **Max throughput**: 250 MB/s, **Max IOPS**: 250.

Key takeaway: **st1 & sc1** are great when you need **low-cost storage** for **high-throughput** or **archival** data.

📚 Summary of EBS Volume Types:

Volume Type	Use Case	Max IOPS	Max Throughput
gp2	General Purpose (boot volumes, dev/test)	3,000 (burstable)	250 MB/s
gp3	Cost-efficient, general purpose	16,000 (independent)	1,000 MB/s
io1	High IOPS for databases, mission-critical	64,000	1,000 MB/s
io2	Ultra high-performance (DBs, I/O intensive)	256,000	1,000 MB/s
st1	Throughput-optimized for big data	500	500 MB/s
sc1	Low-cost, cold storage for archives	250	250 MB/s

🧠 Exam Tips:

- **gp2** and **gp3** are **general-purpose SSDs** suitable for most workloads.
- **io1/io2** are **provisioned IOPS SSDs**, ideal for **high-performance databases**.
- **st1/sc1** are **low-cost HDDs** for **large, throughput-intensive** workloads or **cold storage**.
- **Remember**: **gp3** is the **newer** and **more cost-efficient** version of **gp2**.

⚡ Quick Recap:

- **gp2/gp3**: Balanced performance for general workloads.
- **io1/io2**: High IOPS for mission-critical applications (e.g., DBs).
- **st1/sc1**: Low-cost HDDs for throughput or archival storage.

Visual Aid:

```
                    EBS Volume Types
                ┌──────────────────────────┐
                |       SSD Volumes        |
                ├──────────────────────────┤
                |  - gp2 / gp3 (General)   |
                |  - io1 / io2 (Provisioned)|
                └──────────────────────────┘
                          ↑
   ┌──────────────────────┼──────────────────────┐
   |                      |                      |
┌─────────┐          ┌──────────────┐       ┌─────────┐
| Throughput|        | Provisioned IOPS|     | Cold/Big  |
| Optimized |        | (Mission-Critical)|   | Storage   |
└─────────┘          └──────────────┘       └─────────┘

 st1 (500MB/s)        io1/io2 (64,000 IOPS)     sc1 (Low cost)
```

Here's a breakdown of the **Multi-Attach feature** of **EBS Volumes** for your interview prep. I've added some emojis and key points to make it easier to remember.

📎 Multi-Attach Feature of EBS Volumes

---

1. What is Multi-Attach?

- The **Multi-Attach** feature allows you to attach the **same EBS volume** to **multiple EC2 instances** within the **same availability zone**.
    - **Main Benefit**: Multiple EC2 instances can read from and write to the same volume simultaneously.

---

2. Which EBS Volumes Support Multi-Attach?

- **Supported EBS Types**:
  - Only the **io1** and **io2** volumes support this feature.
  - **Note**: You **cannot** use **gp2**, **gp3**, or other EBS types for Multi-Attach.

---

3. Key Features of Multi-Attach:

- **Multiple EC2 Instances**:
  - The same volume can be **attached to up to 16 EC2 instances** at the same time in a single **availability zone**.
  - **Note**: The limit for Multi-Attach is **16 instances**.
- **Full Read/Write Permissions**:
  - All attached EC2 instances have **full read and write permissions** to the attached io2 volume.
  - **Simultaneous Access**: Multiple EC2 instances can read and write to the same volume at the same time.

---

4. Use Case for Multi-Attach:

- **High Availability for Applications**:
  - Ideal for scenarios requiring **higher application availability**.
  - **Example**: **Clustered Linux applications** like **Teradata** or any application that requires **concurrent write operations**.

---

5. Important Considerations:

- **Availability Zone Restriction**:
  - The **EBS volume** can only be attached to instances **within the same availability zone**.
  - **Cannot attach across AZs**.
- **Cluster-Aware File Systems**:
  - To make Multi-Attach work properly, you must use a **cluster-aware file system**.
    - Examples: **Clustered file systems** like **GFS2** or **OCFS2**.
    - **Note**: **XFS** and **EXT4** file systems are **not cluster-aware**, so they won't work for Multi-Attach.

⚡ Recap: Multi-Attach in a Nutshell

- **Feature**: Attach **one io1/io2 EBS volume** to **multiple EC2 instances** (up to 16) in the **same AZ**.
- **Use Case**: **High availability**, **clustered apps**, and **concurrent write operations**.
- **Limitations**:
  - Works **only in the same AZ**.
  - Must use **cluster-aware file systems**.
  - Limit of **16 instances** per volume.

🔑 Key Exam Points:

- **Supported EBS Types**: **io1** and **io2** (only).
- **Maximum Instances**: **16** EC2 instances per volume.
- **File System**: Must be **cluster-aware** (e.g., GFS2, OCFS2).

Visual Representation:

```
          Multi-Attach Feature Overview

          ┌──────────────────────────┐
          |      io1/io2 EBS Volume  |
          |  Attached to up to 16 EC2 |
          |      Instances in SAME   |
          |   Availability Zone ONLY |
          └──────────────────────────┘

                        ↑
                        |
     ┌──────────────────┼──────────────────┐
     |                  |                  |
  Read/Write       Concurrent Writes   Cluster-Aware FS
 By Multiple EC2   for High Availability    (e.g., GFS2)
  Instances
```

This is a **high-level overview** of the **Multi-Attach** feature. It's essential to understand its **use case**, **limitations**, and **file system requirements** for your exam and interviews. Let me know if you need further clarification! 😊

Here's a breakdown of how to **encrypt EBS volumes**, including the process and key details. I've added emojis and concise points to help you absorb the information quickly for your interview preparation.

🔒 Encrypting EBS Volumes

---

1. Encryption Overview:

- **Automatic Encryption**: When you create an **encrypted EBS volume**, the following are encrypted:
  - **Data at rest** within the volume.
  - **Data in transit** between the instance and the volume.
  - **Snapshots** taken from the volume.
  - **Volumes created from encrypted snapshots**.
- **Transparent Process**:
  - EC2/EBS handles all the encryption and decryption behind the scenes.
  - Minimal impact on **latency** (very little performance hit).
- **Encryption Key**:
  - EBS uses **AES-256 encryption** via **AWS KMS (Key Management Service)**.

---

2. How to Encrypt an Unencrypted EBS Volume:

If you have an **unencrypted EBS volume** and need to encrypt it, follow these steps:

Step-by-Step Process:

1. Create a Snapshot of the Unencrypted Volume:

- Go to **EC2 Console → Volumes**.
- Create a snapshot of the **unencrypted volume**.

  Note: By default, the snapshot will be **unencrypted**.

---

2. Copy the Snapshot and Enable Encryption:

- Go to **Snapshots → Select Snapshot → Actions → Copy Snapshot**.
- In the **Copy Snapshot** screen, select **Enable Encryption**.
  - You can also choose the **KMS Key** for encryption (default or custom key).

---

3. Create a New Encrypted Volume from the Snapshot:

- After the snapshot is copied and encrypted, go to **Snapshots**.
- Select the encrypted snapshot and click **Create Volume**.
- The new volume will automatically be **encrypted**.

---

4. Attach the Encrypted Volume to Your EC2 Instance:

- Attach the newly created **encrypted volume** to the original EC2 instance (or a new one).

---

5. Shortcut Option: Enable Encryption on the Fly:

- Instead of manually copying and then creating a new encrypted volume:
  - From the **Snapshot** page, select the unencrypted snapshot.
  - Go to **Actions → Create Volume**.
  - During the volume creation process, **enable encryption** on the fly. Select the **KMS Key** for encryption.

  Note: This allows you to **encrypt the volume directly** from the snapshot without needing to copy it first.

---

6. Deleting Unused Snapshots and Volumes:

- **Cleanup**: After encryption, remember to delete any unneeded snapshots and volumes to avoid unnecessary costs.

---

7. Key Considerations:

- **Encryption is done via KMS** (AWS Key Management Service), so make sure you have access to the required **KMS keys**.
- **Snapshots** copied from an **unencrypted volume** will initially be unencrypted, so you must enable encryption when copying.
- **Impact on Latency**: Very minimal impact on performance when using encryption.

📚 Key Takeaways:

1. **EBS Encryption** provides **data-at-rest** and **data-in-transit** protection using AES-256 encryption.
2. **Snapshots** are encrypted by enabling encryption during the copy process.
3. **Encrypted volumes** can be created directly from encrypted snapshots.

4. **KMS** (Key Management Service) is used to manage the encryption keys.
5. **Minimal latency** impact when using encryption.

Visual Representation:

```
Encrypting an EBS Volume in Steps

┌──────────────────────────────────┐
|    Unencrypted EBS Volume        |
└──────────────────────────────────┘

            ↓

    Create Snapshot (unencrypted)

            ↓

┌──────────────────────────────────┐
|    Copy Snapshot (Enable Encryption) |
└──────────────────────────────────┘

            ↓

    New Encrypted Snapshot Created

            ↓

┌──────────────────────────────────┐
| Create New Encrypted EBS Volume  |
└──────────────────────────────────┘

            ↓

    Attach Encrypted Volume to EC2 Instance
```

Here are the interview notes for **Amazon EFS (Elastic File System)** based on the transcript you provided:

🔷 What is Amazon EFS?

- **EFS (Elastic File System)** is a **managed NFS (Network File System)** that allows multiple **EC2 instances** (in different availability zones) to mount and access a shared file system simultaneously.
- **Key Features**:
  - **Highly available** and **scalable**.
  - **Cost**: About 3x more expensive than **GP2 EBS** volume. It's **pay-per-use** (you pay for what you use).

- **No upfront capacity planning** needed: EFS scales automatically as data grows.

🔷 Key Use Cases for EFS

1. **Content Management**
2. **Web Serving**
3. **Data Sharing**
4. **WordPress**

   Note: Uses **NFS protocol** internally.

   Security: Set up a **Security Group** to control access.

   Compatibility: Only works with **Linux-based AMI** (not **Windows**).

🔷 EFS Features

- **Encryption at rest**: Uses **KMS (Key Management Service)**.
- **POSIX-compliant**: Standard Linux file system.
- **Standard file API**.

🔷 Performance Modes

- **General Purpose (default):**
  - **Low-latency**, ideal for web servers, CMS, and other latency-sensitive applications.
- **Max I/O Mode:**
  - **High-latency** but **higher throughput**.
  - Ideal for **big data** applications or **media processing**.
  - **Highly parallel** processing.

🔷 Throughput Modes

1. **Bursting Throughput**:
   - **Base throughput**: 50 MB/s.
   - **Bursting**: Up to 100 MB/s based on usage.
2. **Provisioned Throughput**:
   - You can set throughput independently of storage size (e.g., 1 GB/s for 1 TB storage).

3. **Elastic Throughput**:
    - Automatically adjusts throughput based on workload demand.
    - Example: 3 GB/s for reads and 1 GB/s for writes, perfect for **unpredictable workloads**.

🔷 Storage Classes

1. **Standard**:
    - **For frequently accessed data**.
    - Recommended for **production workloads** and multi-AZ setups.
2. **EFS-IA (Infrequent Access)**:
    - **Cost-effective** for data that's **rarely accessed**.
    - **Retrieval cost** is higher, but storage cost is lower.
3. **Archive Storage**:
    - **Very low cost** for data that is **accessed only a few times a year**.

🔷 Lifecycle Management

- Automatically move files between different storage tiers based on access frequency.
- **Example**: After 60 days of non-access, move a file from **EFS Standard** to **EFS-IA**.

  📅 **How Lifecycle Works**:

```
+-------------------------+
|      EFS Standard        |
+-------------------------+
              ↓
  (File not accessed in 60 days)
              ↓
+-------------------------+
|       EFS-IA (Infrequent Access)   |
+-------------------------+
```

🔷 Availability & Durability

1. **Multi-AZ Setup (Standard):**
    - Resilient to **disasters**.

    - Ideal for **production** environments.
2. **Single-AZ Setup (One Zone)**:
    - Cheaper for **development**.
    - Still offers **backups**.
    - Compatible with **IA storage**.

🔷 Cost Savings

- **Up to 90%** cost savings with the **right storage class**.
    - Use **EFS-IA** and **Archive Storage** for infrequently accessed or rarely accessed data.

🔷 Summary

- **Amazon EFS** is a flexible, scalable, and high-performance file system that is ideal for **Linux-based applications**.
- Use cases include **content management**, **web serving**, and **big data**.
- Offers different **performance modes** (General Purpose & Max I/O) and **throughput modes** (Bursts, Provisioned, Elastic).
- **Lifecycle policies** allow moving files to cheaper storage tiers based on access frequency.

💡 Quick Recap for Interview:

1. **What is EFS?**
    - Managed NFS, highly scalable, used across multiple AZs.
2. **Key Features:**
    - Scalable, pay-per-use, no capacity planning.
    - Only for Linux-based AMI.
3. **Performance & Storage:**
    - General Purpose vs Max I/O (low latency vs high throughput).
    - Bursting, Provisioned, and Elastic throughput options.
4. **Storage Classes:**
    - Standard, Infrequent Access (EFS-IA), Archive storage.
5. **Lifecycle & Cost Optimization:**
    - Automate file tiering, **up to 90% savings**.

These notes should help in a structured and concise way for your interview prep. Do you need any more details or diagrams for specific topics? 😊

Here's a comprehensive breakdown of the hands-on practice for **Amazon Elastic File System (EFS)**, ideal for interview prep:

🔷 Creating an Amazon EFS File System

1. **Navigate to EFS Console**:
    - Choose a **VPC** for the file system.
    - **Optional Name**: You can provide a name or leave it empty.
2. **File System Type**:
    - **Regional** (Default) = **Multi-AZ** setup for **high availability and durability**.
    - **One Zone** = Single AZ, cheaper, suitable for **development environments**, but **not ideal for production**.
3. **Enable Automatic Backups** (recommended):
    - Ensures data safety and recovery.
4. **Lifecycle Management** (for cost-saving):
    - Transition files to cheaper tiers:
        - **Infrequent Access (EFS-IA)**: Files not accessed for **30 days**.
        - **Archive**: Files not accessed for **90 days**.
        - **Revert to Standard** once the file is accessed.
    - 🔄 **Lifecycle flow**:

      `Standard → IA → Archive → Access → Standard`
5. **Encryption**: Enable **KMS encryption** for security at rest.
6. **Performance Settings**:
    - **Throughput Modes**:
        - **Bursting**: Scales throughput with storage.
        - **Elastic** (recommended): Auto-scales based on workload; **best for unpredictable I/O**. No need to manage throughput settings.
        - **Provisioned**: Set throughput **in advance**, pay for what you provision.
    - **Performance Modes**:
        - **General Purpose** (Low latency, good for **latency-sensitive applications**).

- **Max I/O** (Higher latency, best for **high-parallelized workloads** like **big data**).

🔷 Network Access Settings

- Choose a **VPC** and configure **mount targets**.
- For **Regional setup**, assign **multiple AZs** with subnets.
- **Security Groups**:
  - Create and attach a security group for access control.

🔷 Mounting EFS to EC2 Instances

1. **Launch EC2 Instances**:
   - Instance A in **AZ A** and Instance B in **AZ B**.
   - Use **Amazon Linux 2** (Free Tier eligible).
   - **No Key Pair**: Use **EC2 Instance Connect** for access.
2. **EFS Setup for EC2 Instances**:
   - Configure **EFS mount points** within EC2 console.
   - **Automatic Security Groups**: EC2 auto-creates security groups for **EFS** access.
3. **File System Mounting**:
   - Instance A mounts EFS at `/mnt/efs/fs1`.
   - Instance B automatically mounts the same EFS.

🔷 File Creation & Access Between EC2 Instances

1. **Write to EFS from Instance A**:
   - **Commands**:

     ```
     sudo su
     echo "hello world" > /mnt/efs/fs1/hello.txt
     cat /mnt/efs/fs1/hello.txt
     ```

     - Result: `hello world`.
2. **Access EFS from Instance B**:
   - Check if `hello.txt` exists:

     ```
     ls /mnt/efs/fs1/
     cat /mnt/efs/fs1/hello.txt
     ```

- **Output**: `hello world` on both EC2 instances.

Key Concept: Both EC2 instances (in different AZs) can **share the same EFS** seamlessly. The file system is mounted on both and accessible from both.

🔷 Security Groups & Auto-Creation

- When you launch EC2 instances and mount them to EFS, the EC2 console **auto-creates security groups** (e.g., `efs-sg-1`, `efs-sg-2`) for EFS access.
- The security group **allows NFS** protocol on port **2049** for EC2 instances to communicate with EFS.

🔷 Clean-up Steps

1. **Terminate EC2 Instances**: Ensure instances are properly terminated.
2. **Delete EFS**: Go to the **EFS Console**, enter file system ID, and delete it.
3. **Remove Extra Security Groups**: Clean up security groups created during the demo.

💡 Key Takeaways for Interview

- **EFS File System Types**:
  - **Regional** = **Multi-AZ**, high availability.
  - **One Zone** = Single AZ, cheaper, dev environments.
- **Lifecycle Management**:
  - Transition files to **Infrequent Access** (IA) or **Archive** for cost savings.
- **Performance Modes**:
  - **Elastic** (recommended) = Auto-scaling throughput, great for unpredictable workloads.
  - **Bursting** = Throughput scales with storage.
  - **Provisioned** = Predefined throughput for consistent performance needs.
- **Security & Network**:
  - Attach custom **security groups** to ensure access.
  - Mount EFS to EC2 instances with **automatic configurations** in the console.

📝 Final Diagram Recap (for better clarity)

```
+-------------------+       +--------------------+
|  EC2 Instance A   |       |   EC2 Instance B   |
|   (AZ A)          | <---> |   (AZ B)           |
+-------------------+       +--------------------+
         |                          |
         |  Mounts /mnt/efs/fs1/    |
         v                          v
   +-----------------------------+
   |        EFS File System      |
   |   (Regional, Multi-AZ, NFS) |
   +-----------------------------+
         ↕
     Shared Files
```

Here are the detailed **interview notes** on the **differences between EBS** (Elastic Block Store), **EFS** (Elastic File System), and **Instance Store** based on your provided transcript:

🔷 Key Differences Between EBS, EFS, and Instance Store

🔶 Amazon EBS (Elastic Block Store)

- **Attachment**:
  - Can only be attached to **one EC2 instance** at a time.
  - **Multi-Attach Feature** (io1 & io2 volumes) allows multiple EC2 instances to attach to an EBS volume, but this is **for specific use cases**.
- **Availability Zone (AZ)**:
  - EBS volumes are tied to a **specific AZ**.
  - Example: An EBS volume in **AZ 1** cannot be attached to an EC2 instance in **AZ 2**.
- **IO (Input/Output) Behavior**:
  - **gp2**: IO scales with the **volume size** (larger volumes = more IOPS).
  - **gp3 & io1**: You can independently scale **IOPS** (IO operations per second) and **volume size**.
- **Snapshot & Migration**:

    - To migrate an EBS volume between AZs, you must take a **snapshot** and then restore it in another AZ.
    - **EBS Snapshots** can impact **IO performance**, so it's best to run them during low traffic periods.
- **Backup Behavior**:
    - EBS **backups use IO**, which can affect application performance during heavy traffic.
- **Termination Behavior**:
    - By default, **EBS volumes are terminated** when the EC2 instance they're attached to is terminated, but this can be **disabled**.

◆ Amazon EFS (Elastic File System)

- **Network File System**:
    - Designed for **multiple EC2 instances** across **multiple availability zones (AZs)**.
    - Great for shared data needs (e.g., **WordPress**), especially for **Linux-based instances** using the **POSIX** file system.
- **Multiple EC2 Instances**:
    - EFS can be mounted to hundreds of **EC2 instances** at once across different AZs, enabling **simultaneous access** to the same file system.
- **Scalability**:
    - Automatically scales as data grows.
    - Ideal for use cases requiring **shared file storage** with **low-latency access**.
- **Price Point**:
    - **Higher cost** compared to EBS, but cost savings can be achieved by using **storage tiers** (e.g., **EFS-IA** and **Archive Storage**).
- **Flexibility**:
    - Supports **NFS protocol** for easy file sharing.
    - Primarily designed for use with **Linux-based AMIs** (not compatible with Windows).

◆ Instance Store

- **Physical Attachment**:
    - **Physically attached** to the EC2 instance, meaning **if the EC2 instance is terminated**, the data on the instance store is **lost**.

- **Ephemeral Storage**:
  - Used for **temporary** storage that **does not persist** beyond the life of the instance.
  - Common for applications that do not require persistent data or for use cases like **caching**.

💡 Summary of Differences

Feature	EBS (Elastic Block Store)	EFS (Elastic File System)	Instance Store
Attachment	1 EC2 instance at a time (multi-attach for io1/io2)	Multiple EC2 instances across AZs	Physically attached to EC2 instance
Availability Zone	Tied to a specific AZ	Multi-AZ (Regional)	Tied to EC2 instance
IO Scaling	gp2: scales with volume size, gp3/io1: scales independently	Auto-scaling throughput based on workload	Not applicable
Snapshots & Migration	Requires snapshot to migrate across AZs	Not required for multi-AZ access	No snapshot or migration capabilities
Backup Impact	IO-based backups can affect performance	No backup impact due to scaling nature	No persistent storage, data lost with EC2
Cost	Lower than EFS, scales with volume size	Higher cost but with **storage tiers** for savings	Free, but **non-persistent**
Data Persistence	Data persists even if EC2 is terminated	Data persists across EC2 terminations	Data lost when EC2 instance is terminated
Use Cases	Block storage for single EC2 instances or databases	Shared file storage for multiple EC2 instances	Temporary data storage (ephemeral)

🚀 Key Concepts for Interview:

1. **EBS**:
   - Best for **single-instance** storage or block-level data.
   - **Snapshot**-based migration across AZs.
   - **Performance impact** when backing up under heavy load.
2. **EFS**:
   - Designed for **shared access** across **multiple EC2 instances** across **AZs**.
   - **Auto-scaling**, **pay-per-use**.
   - Best for **Linux-based applications** requiring file sharing (e.g., **WordPress**).
3. **Instance Store**:
   - Ideal for **ephemeral data** (temporary).
   - **No persistence**; lost if EC2 is terminated.

Here’s a **clean-up checklist** to ensure everything is properly deleted and you're not left with any unused resources, perfect for maintaining cost efficiency and tidying up after a project:

🔷 Clean-Up Steps in AWS:

---

1. Delete EFS File System

- Go to **EFS Console** → Select the **file system** you want to delete.
- **Action** → **Delete File System**.
- Enter the **File System ID** to confirm the deletion.
- Once deleted, the **EFS** is removed and you will stop paying for the storage.

---

2. Terminate EC2 Instances

- **Terminate all EC2 instances** that were running:
  - Go to **EC2 Console** → **Instances** → Select instances to terminate → **Terminate**.
  - Ensure instances are **properly terminated** before proceeding.

---

3. Delete EBS Volumes

- EBS volumes associated with terminated instances must also be deleted:
  - **EBS Volumes Console** → **Select volumes** → **Right-click** → **Delete Volume**.
  - Deleting volumes ensures you are not charged for any unused storage.

---

4. Delete Snapshots

- Snapshots take up storage and can incur charges, so you should delete them:
  - Go to **Snapshots** in the **EC2 Console**.
  - Select and **delete** all snapshots that you no longer need.
  - This ensures you're not paying for backup storage that you don’t require.

---

5. Delete Security Groups

- **Security Groups Cleanup**:
  - **EC2 Console** → **Security Groups** → **Delete unused security groups**.
  - **Important**: Do **not delete the default security group** as it’s required by the system.

- **Delete any custom security groups** that were associated with terminated EC2 instances or services:
    - If a security group is still in use (attached to any running instance), you'll need to wait until the instance is terminated and the association is cleared before deleting the security group.
- **Tip**: Keep checking periodically until all security groups are deleted. Some may not delete immediately if they are still in use.

🔷 Final Cleanup Steps

- **Elastic Load Balancers (ELB)**: If any load balancers were created, ensure they are deleted as well.
    - **EC2 Console → Load Balancers → Delete** any unnecessary ones.

🧹 Summary of the Cleanup Process:

1. **Delete File Systems**: EFS file system and associated mounts.
2. **Terminate EC2 Instances**: Ensure instances are shut down and terminated properly.
3. **Delete Volumes**: Remove unused EBS volumes.
4. **Delete Snapshots**: Clear any old snapshots to avoid storage costs.
5. **Delete Security Groups**: Ensure only necessary security groups remain, and delete unused ones.
6. **Check Load Balancers**: If applicable, remove any unnecessary load balancers.

💡 Key Takeaways:

- **Cost Management**: Regular clean-up helps manage and reduce unnecessary AWS costs, especially from unused resources like **EBS volumes** and **snapshots**.
- **Security**: Ensure unused **security groups** are removed to prevent accidental exposure or conflicts in future projects.